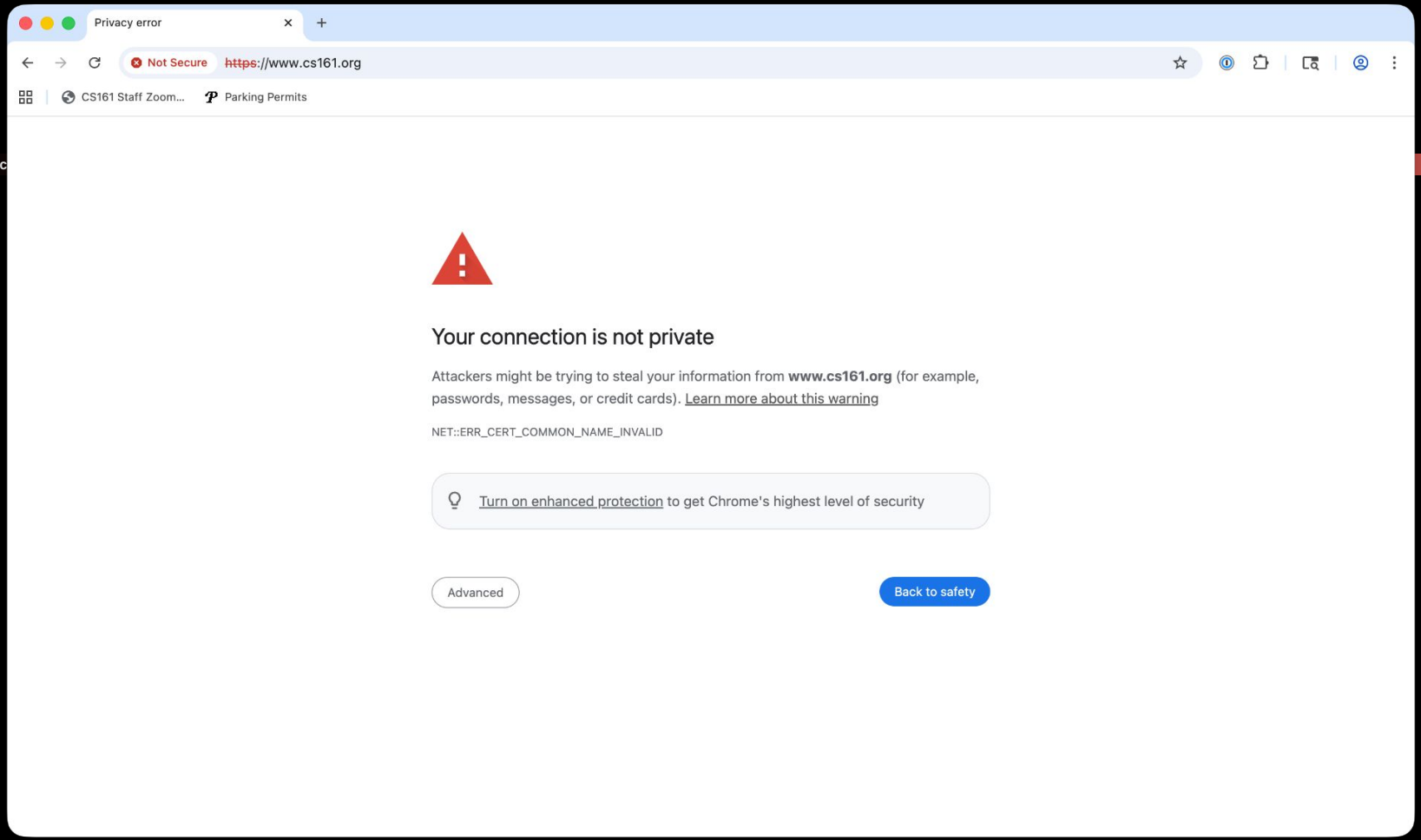
TLS

CS 161 Spring 2026 - Lecture 29

(Textbook Chapter 31)



Computer Scienc



My goals for the next two lectures

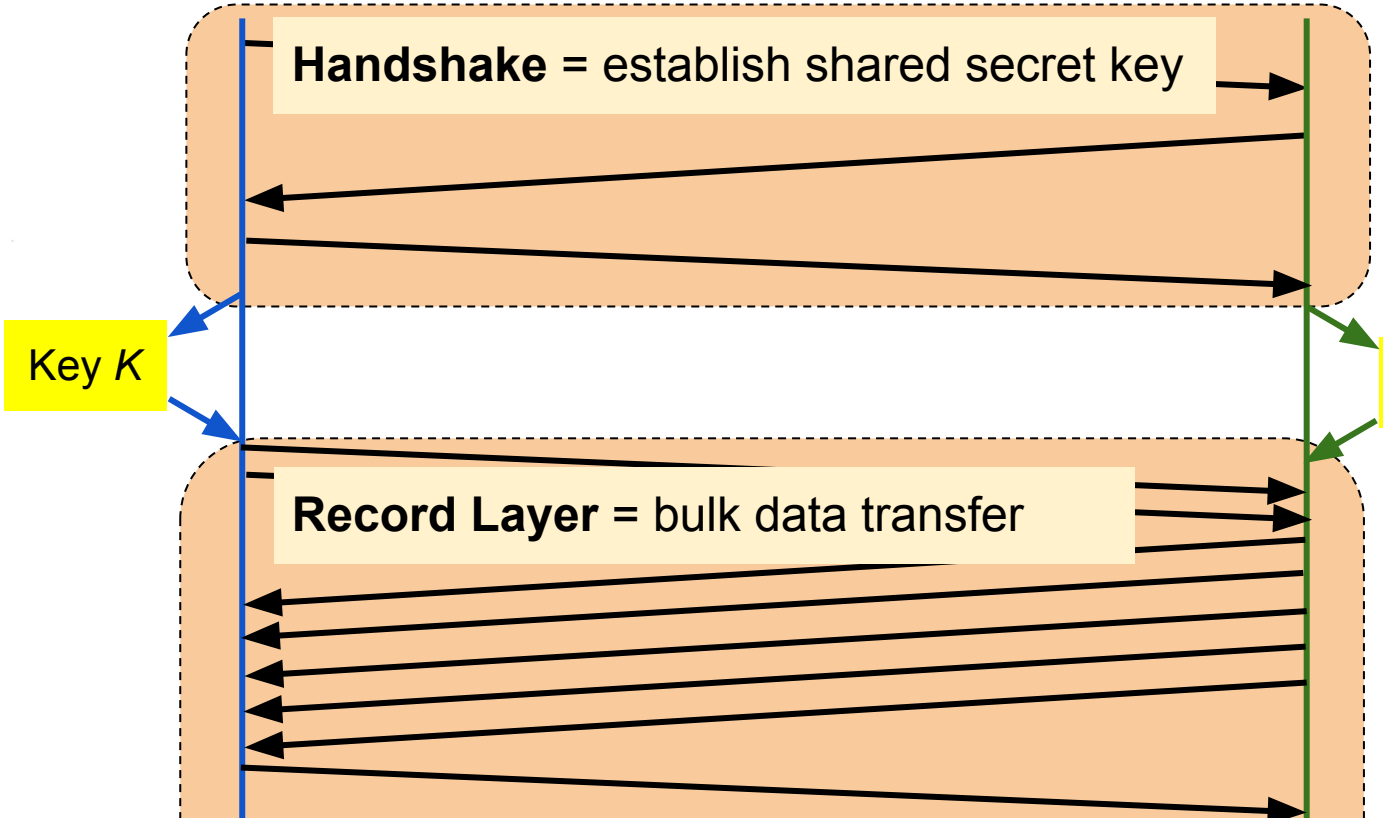
- Give you enough background on TLS to use it wisely
  - What it does, what its limitations are
- Give you an appreciation for the subtlety of protocol design
  - Via examples of what can go wrong
  - Even coming up with the right security properties is tricky
- Deter you from trying to cook up your own transport-security protocols
  - Unless you want to first devote many years of your life to studying protocol analysis

Will focus on TLS version 1.3, the latest version

TLS has two sub-protocols







Plan for today

- What is TLS for?
- Cryptography refresher
  - Public-key certificates
- TLS handshake
  - Simplified version
  - What can wrong
- TLS record layer

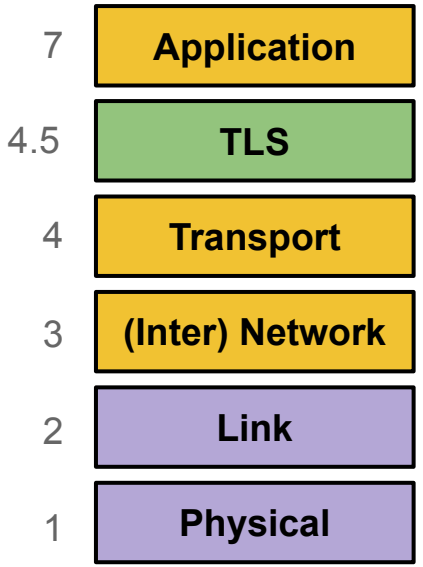
TLS



Textbook Chapter 31

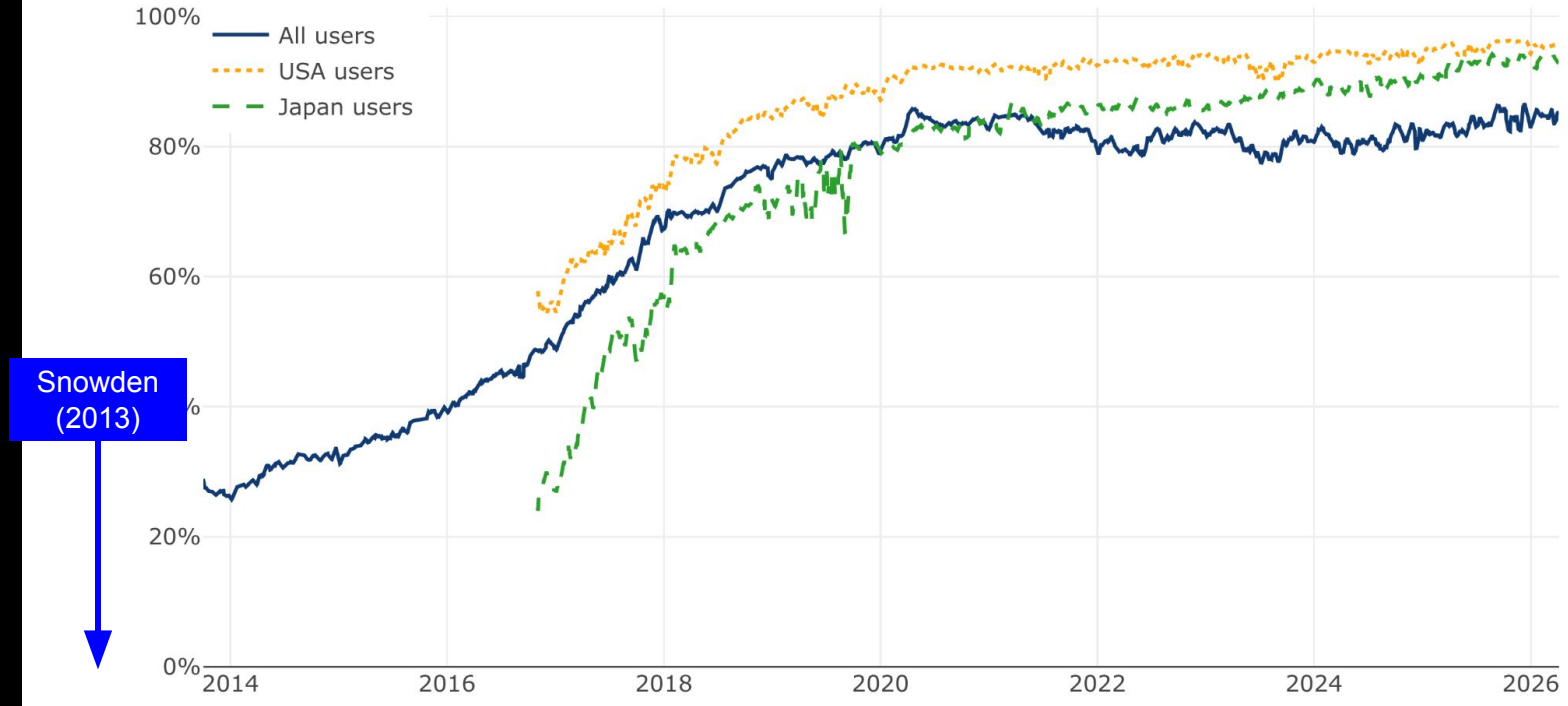
Transport-Layer Security (TLS)

- TLS: A protocol for creating an **encrypted and authenticated** communication channel over the Internet
  - Replaces SSL (Secure Sockets Layer)
- **Functionality goal:** Be a drop-in replacement for TCP
  - Any application that runs over TCP can run over TLS
  - Distinguishes TLS from, SSH or Signal, which both provide end-to-end encryption but are not drop-in replacements for TCP
- TLS is a critical piece of Internet infrastructure
  - HTTPS = HTTP over TLS
  - Also used for mail (IMAPS, POPS, SMTPS), phone apps, etc.
  - The first tool to reach for when sending data over the Internet
  - **Using TLS defends against many lower-level network attacks**



Percentage of Web Pages Loaded by Firefox Using HTTPS

(14-day moving average, source: Firefox Telemetry)



https://letsencrypt.org/stats/

Cryptography Refresher

Recall: Diffie-Hellman key exchange

Public parameters: – An n-bit prime p // "modulus," often 2048 bits long
– A number g in $\{0, \ldots, p - 1\}$ // "generator"

Alice's sk_A
Random $a \in \{0, \ldots, p\text{-}1\}$

Bob's sk_B
Random $b \in \{0, \ldots, p - 1\}$



$pk_A = g^a \bmod p$



$pk_B = g^b \bmod p$

$K = (pk_B)^a = (g^b)^a = g^{ab}$

$K = (pk_A)^b = (g^a)^b = g^{ab}$

Recall: Digital signatures

Three efficient algorithms:

- KeyGen() → (*SK*, *PK*) // Generate keypair.
- Sign(*SK*, *M*) → **σ** // Sign with secret key
- Verify(*PK*, *M*, **σ**) → {0,1} // Verify with public key

Properties:

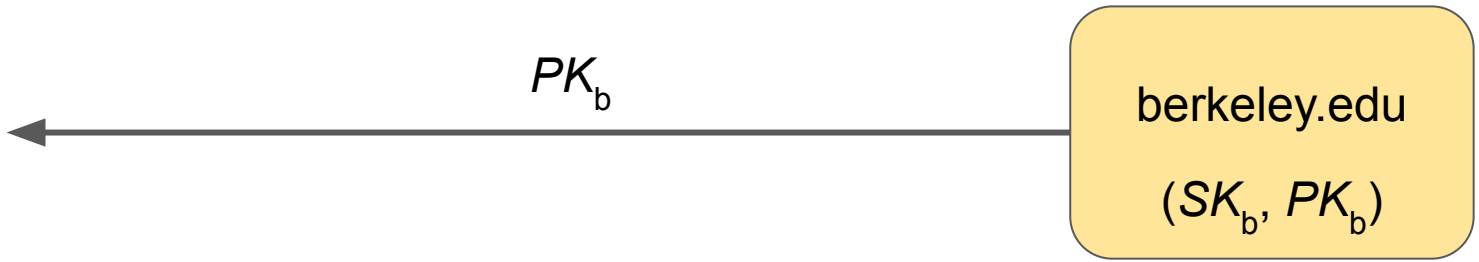
- **Correctness**: If signer signs, verifier accepts
  - For all (*SK, PK*) ← KeyGen() and messages *M*, we have: Verify(*PK*, *M*, Sign(*SK*, *M*)) = 1
- **Security**:
  – Attacker gets public key and signatures on many messages of its choice
  – Still, hard for attacker to produce a valid signature on a *new* message

Public-key certificates

Q: How does Alice know that PK_b is berkeley.edu's real public key?

A: Public-key certificates. → These play a central role in TLS.





Public-key certificates

Certificate authority

(SK_{CA}, PK_{CA})

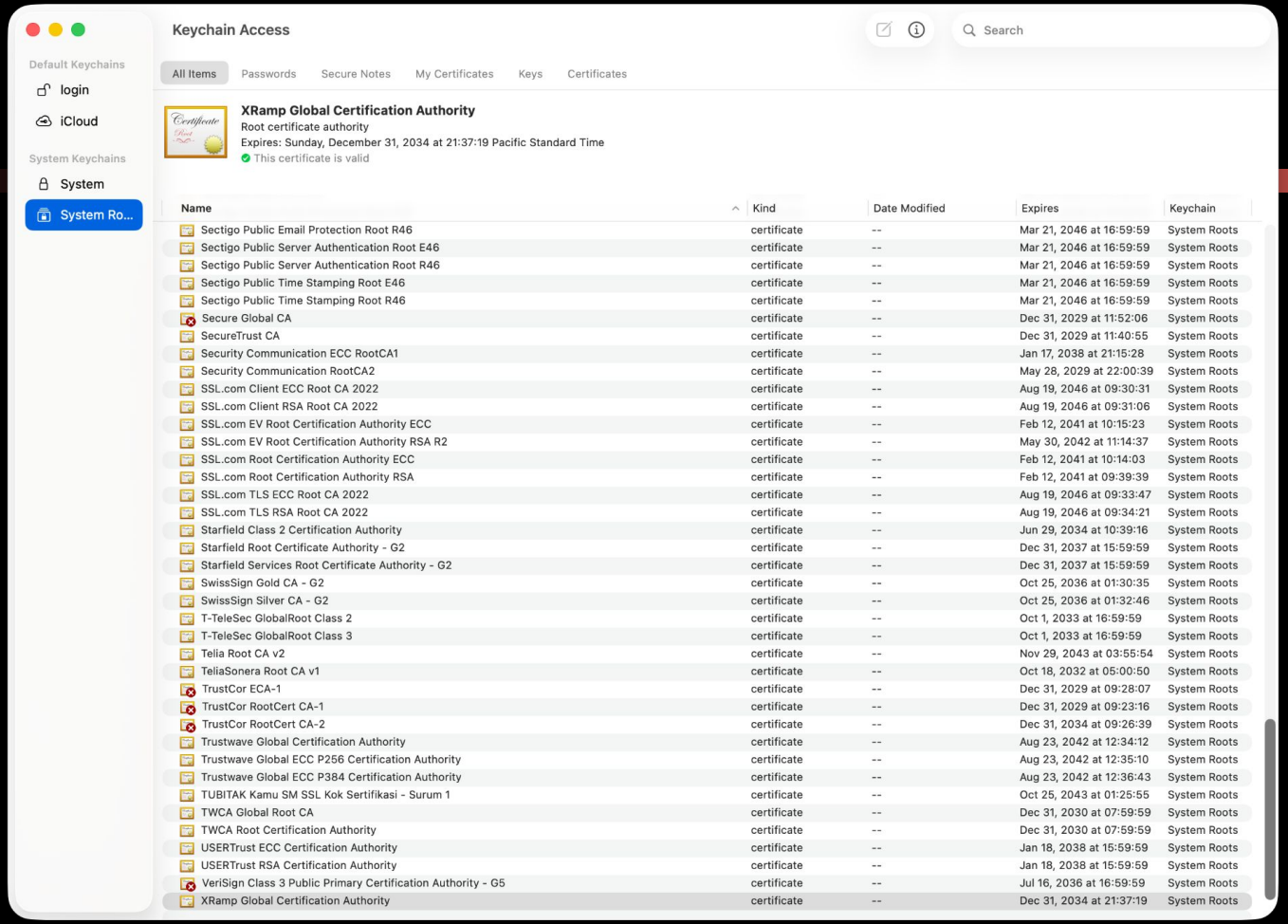


PK_{CA}

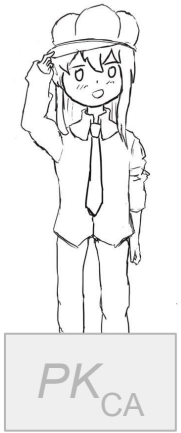
Ships with Alice's OS, browser, or TLS library

berkeley.edu

(SK_b, PK_b)


Keychain Access
Search
Default Keychains
login
iCloud
System Keychains
System
System Ro...
All Items
Passwords
Secure Notes
My Certificates
Keys
Certificates
XRamp Global Certification Authority
Root certificate authority
Expires: Sunday, December 31, 2034 at 21:37:19 Pacific Standard Time
This certificate is valid
Name
Kind
Date Modified
Expires
Keychain
Sectigo Public Email Protection Root R46 certificate -- Mar 21, 2046 at 16:59:59 System Roots
Sectigo Public Server Authentication Root E46 certificate -- Mar 21, 2046 at 16:59:59 System Roots
Sectigo Public Server Authentication Root R46 certificate -- Mar 21, 2046 at 16:59:59 System Roots
Sectigo Public Time Stamping Root E46 certificate -- Mar 21, 2046 at 16:59:59 System Roots
Sectigo Public Time Stamping Root R46 certificate -- Mar 21, 2046 at 16:59:59 System Roots
Secure Global CA certificate -- Dec 31, 2029 at 11:52:06 System Roots
SecureTrust CA certificate -- Dec 31, 2029 at 11:40:55 System Roots
Security Communication ECC RootCA1 certificate -- Jan 17, 2038 at 21:15:28 System Roots
Security Communication RootCA2 certificate -- May 28, 2029 at 22:00:39 System Roots
SSL.com Client ECC Root CA 2022 certificate -- Aug 19, 2046 at 09:30:31 System Roots
SSL.com Client RSA Root CA 2022 certificate -- Aug 19, 2046 at 09:31:06 System Roots
SSL.com EV Root Certification Authority ECC certificate -- Feb 12, 2041 at 10:15:23 System Roots
SSL.com EV Root Certification Authority RSA R2 certificate -- May 30, 2042 at 11:14:37 System Roots
SSL.com Root Certification Authority ECC certificate -- Feb 12, 2041 at 10:14:03 System Roots
SSL.com Root Certification Authority RSA certificate -- Feb 12, 2041 at 09:39:39 System Roots
SSL.com TLS ECC Root CA 2022 certificate -- Aug 19, 2046 at 09:33:47 System Roots
SSL.com TLS RSA Root CA 2022 certificate -- Aug 19, 2046 at 09:34:21 System Roots
Starfield Class 2 Certification Authority certificate -- Jun 29, 2034 at 10:39:16 System Roots
Starfield Root Certificate Authority - G2 certificate -- Dec 31, 2037 at 15:59:59 System Roots
Starfield Services Root Certificate Authority - G2 certificate -- Dec 31, 2037 at 15:59:59 System Roots
SwissSign Gold CA - G2 certificate -- Oct 25, 2036 at 01:30:35 System Roots
SwissSign Silver CA - G2 certificate -- Oct 25, 2036 at 01:32:46 System Roots
T-TeleSec GlobalRoot Class 2 certificate -- Oct 1, 2033 at 16:59:59 System Roots
T-TeleSec GlobalRoot Class 3 certificate -- Oct 1, 2033 at 16:59:59 System Roots
Telia Root CA v2 certificate -- Nov 29, 2043 at 03:55:54 System Roots
TeliaSonera Root CA v1 certificate -- Oct 18, 2032 at 05:00:50 System Roots
TrustCor ECA-1 certificate -- Dec 31, 2029 at 09:28:07 System Roots
TrustCor RootCert CA-1 certificate -- Dec 31, 2029 at 09:23:16 System Roots
TrustCor RootCert CA-2 certificate -- Dec 31, 2034 at 09:26:39 System Roots
Trustwave Global Certification Authority certificate -- Aug 23, 2042 at 12:34:12 System Roots
Trustwave Global ECC P256 Certification Authority certificate -- Aug 23, 2042 at 12:35:10 System Roots
Trustwave Global ECC P384 Certification Authority certificate -- Aug 23, 2042 at 12:36:43 System Roots
TUBITAK Kamu SM SSL Kok Sertifikasi - Surum 1 certificate -- Oct 25, 2043 at 01:25:55 System Roots
TWCA Global Root CA certificate -- Dec 31, 2030 at 07:59:59 System Roots
TWCA Root Certification Authority certificate -- Dec 31, 2030 at 07:59:59 System Roots
USERTrust ECC Certification Authority certificate -- Jan 18, 2038 at 15:59:59 System Roots
USERTrust RSA Certification Authority certificate -- Jan 18, 2038 at 15:59:59 System Roots
VeriSign Class 3 Public Primary Certification Authority - G5 certificate -- Jul 16, 2036 at 16:59:59 System Roots
XRamp Global Certification Authority certificate -- Dec 31, 2034 at 21:37:19 System Roots

Public-key certificates: Certificate generation (rare)



Certificate authority

(SK_{CA}, PK_{CA})

PK_b, proof that I own berkeley.edu domain

$\sigma \leftarrow \text{Sign}(SK_{CA}, \text{“berkeley.edu owns } PK_b\text{”})$

berkeley.edu

(SK_b, PK_b)

Public-key certificates

Certificate authority

(SK_{CA}, PK_{CA})



PK_{CA}

Certificate

(σ, "berkeley.edu owns PK_b")

berkeley.edu

(SK_b, PK_b)

Public-key certificates

Certificate authority

(SK_{CA}, PK_{CA})



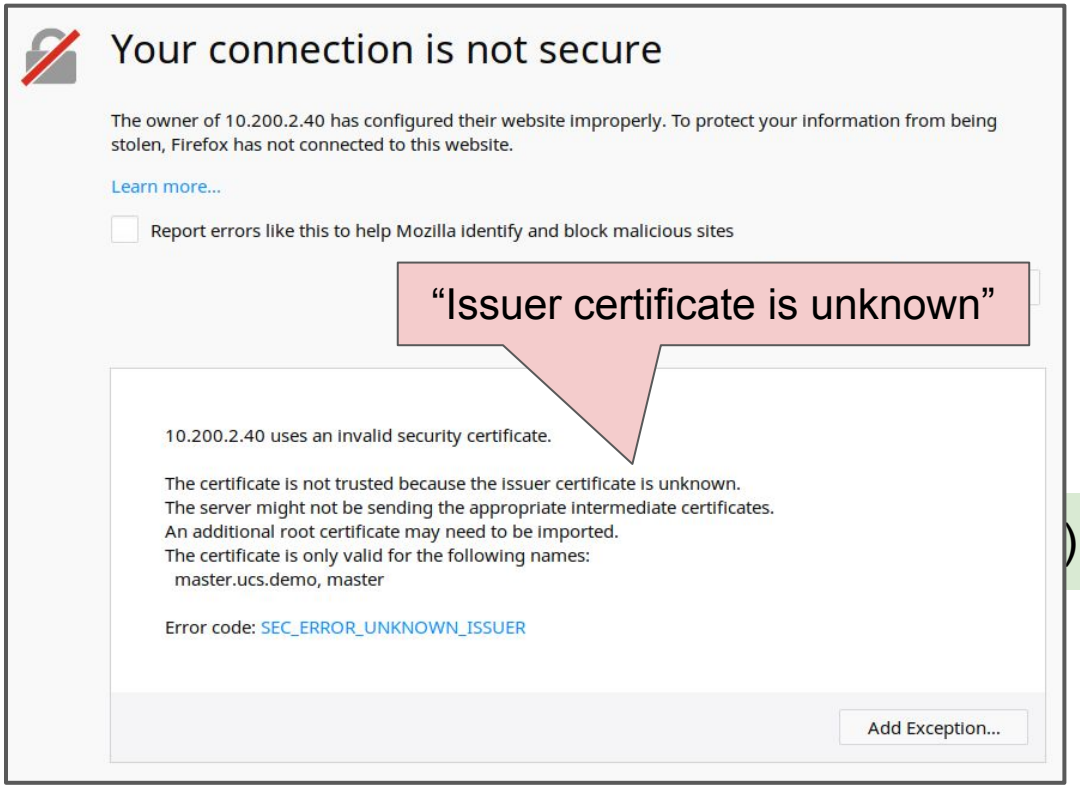
PK_{CA}

Verify(PK_{CA}, "berkeley.edu owns PK_b", σ) = 1 **?**

berkeley.edu

(SK_b, PK_b)

Sometimes you get a certificate from an unfamiliar CA

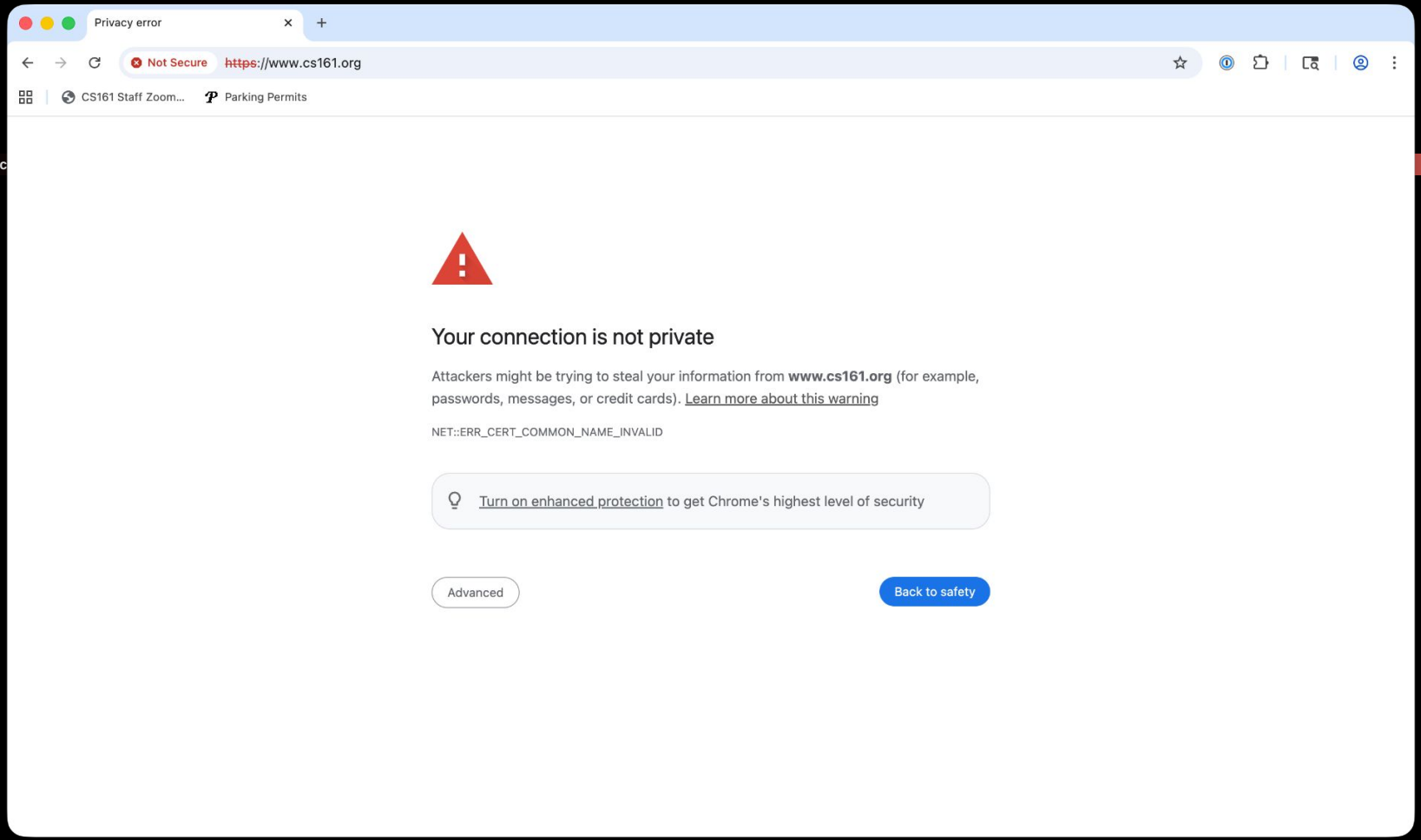


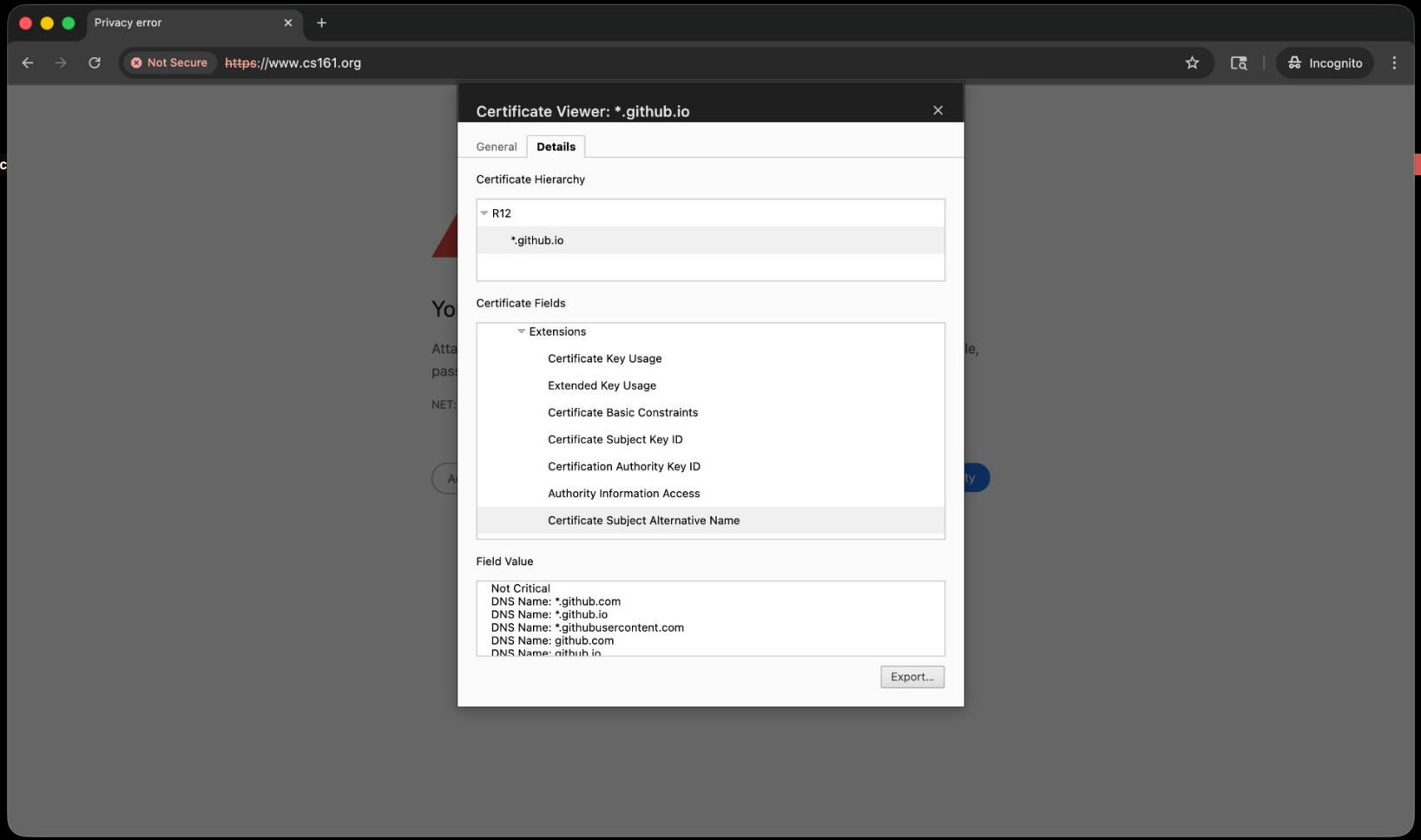
) = 1 ?

berkeley.edu

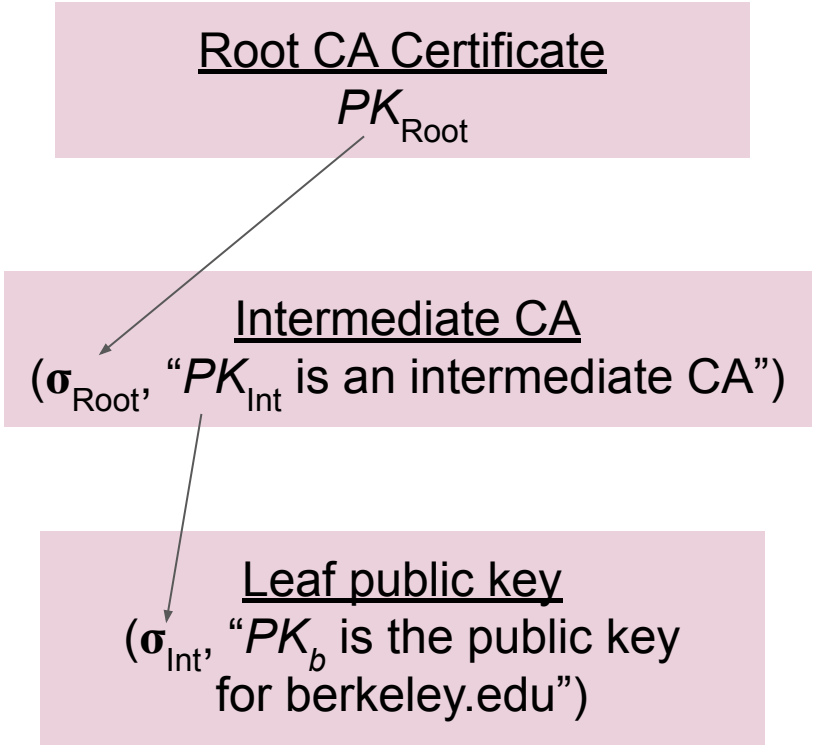
(SK_b, PK_b)

Computer Scienc

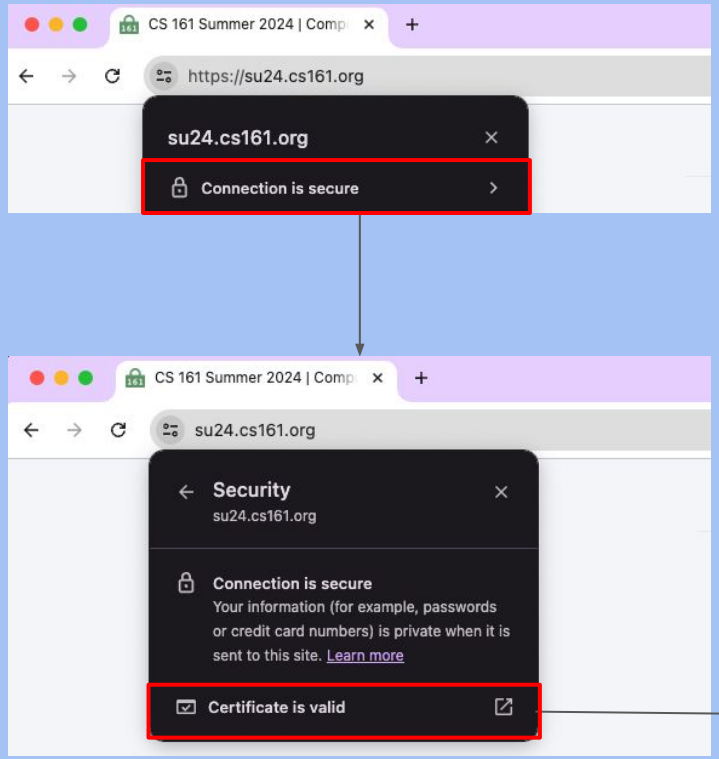


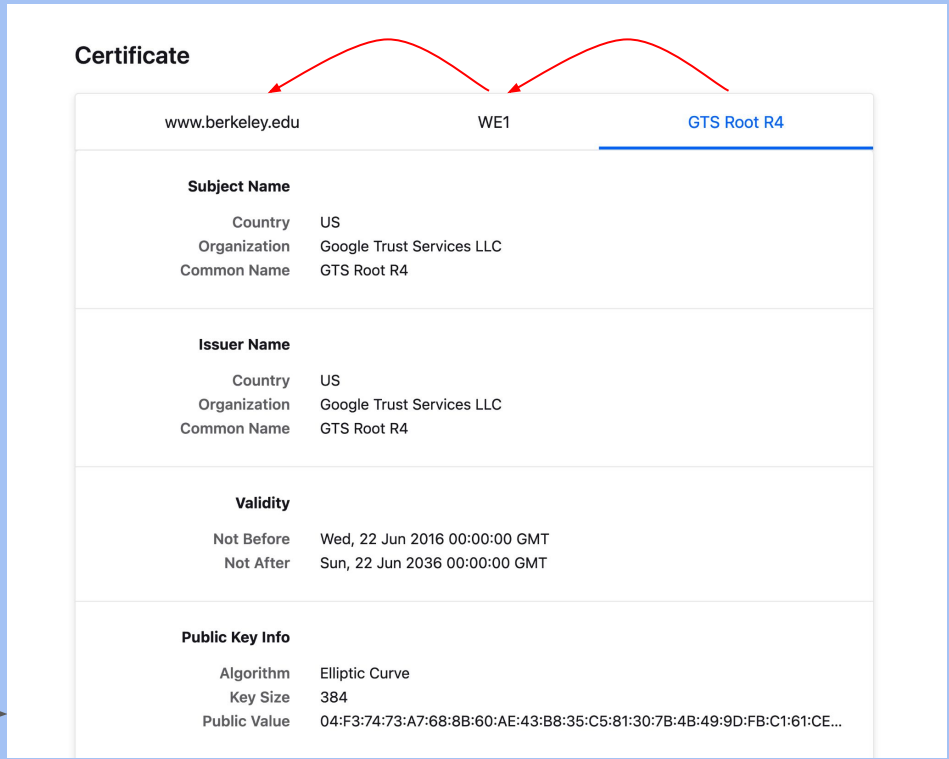

Privacy error
Not Secure
https://www.cs161.org
Incognito
Certificate Viewer: *.github.io
General
Details
Certificate Hierarchy
R12
*.github.io
Certificate Fields
Extensions
Certificate Key Usage
Extended Key Usage
Certificate Basic Constraints
Certificate Subject Key ID
Certification Authority Key ID
Authority Information Access
Certificate Subject Alternative Name
Field Value
Not Critical
DNS Name: *.github.com
DNS Name: *.github.io
DNS Name: *.githubusercontent.com
DNS Name: github.com
Export...

Certificate chains



You can inspect certificate chains in your web browser





TLS Handshake



Many of these slides are based on Dan Boneh's slides on authenticated key exchange.

TLS has two sub-protocols





Handshake = establish shared secret key

Key *K*

Key *K*

Record Layer = bulk data transfer

TLS Handshake: Basic security goals (very informal)

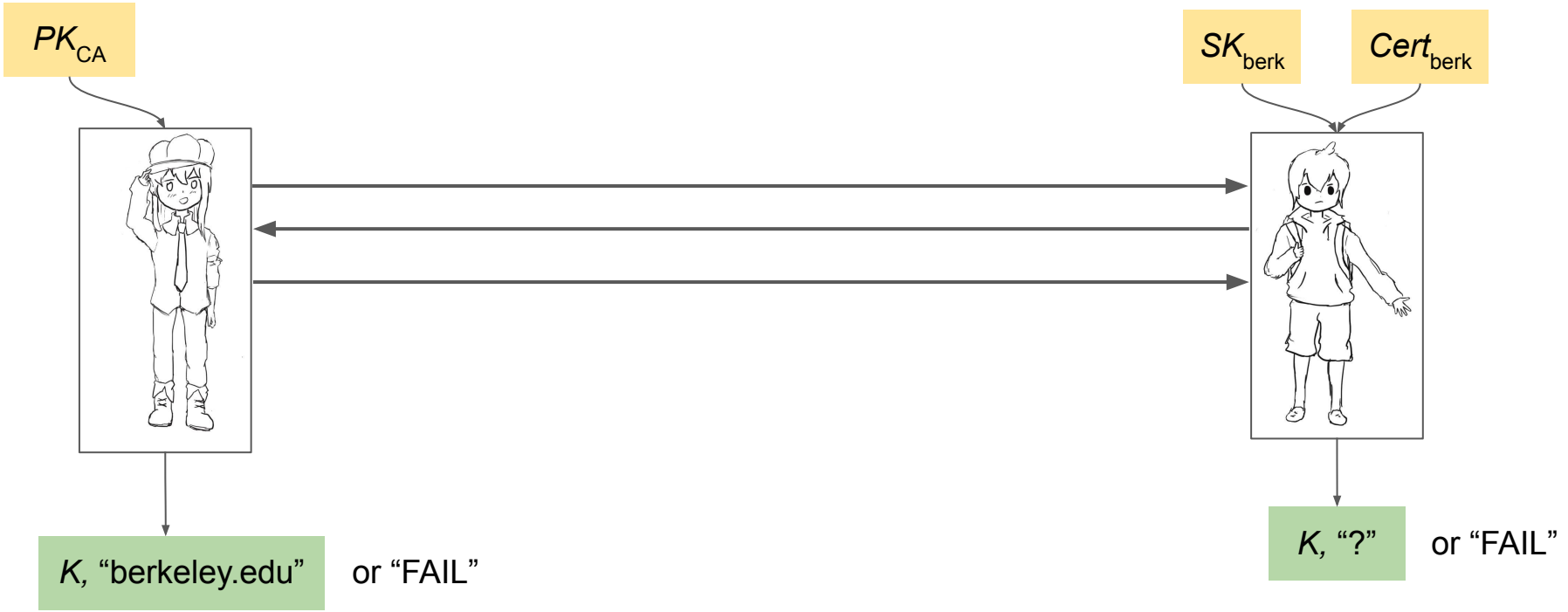
Correctness: Both parties agree on the same session key at the conclusion of the handshake.

Security: Adversary "learns nothing" about the secret key; Parties agree on a fresh random session key each time.

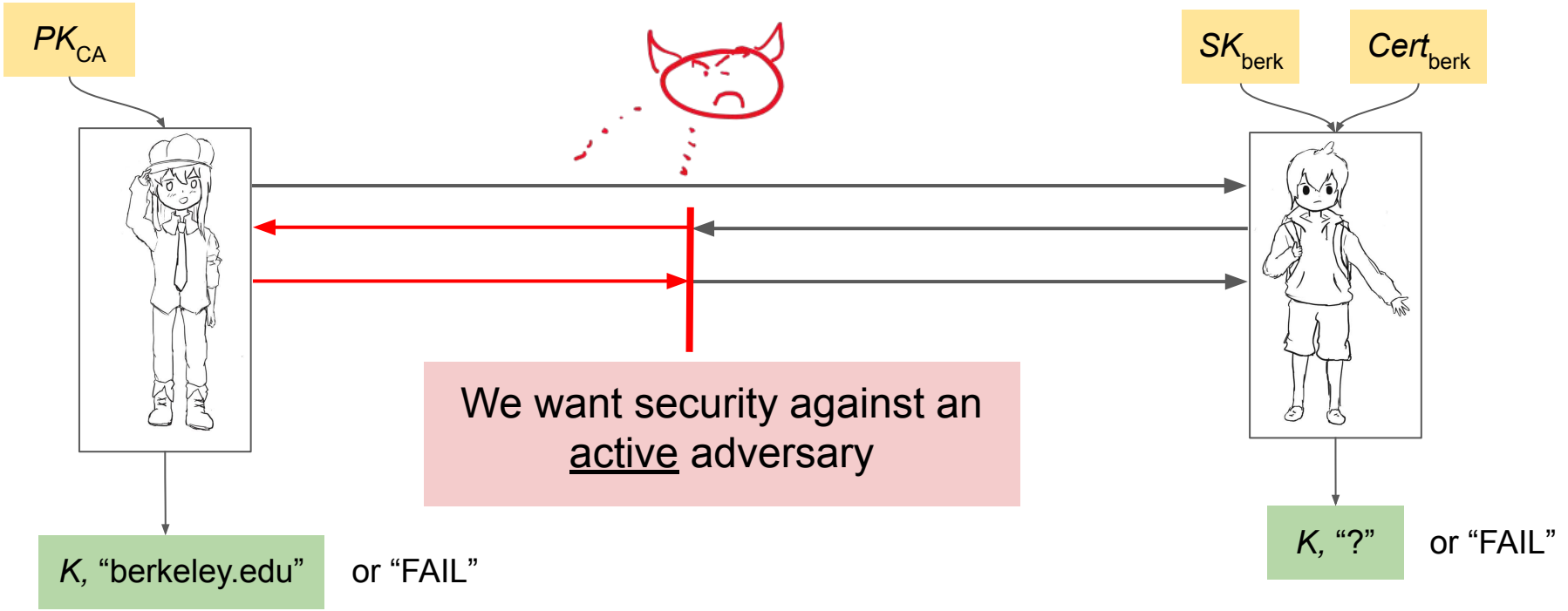
Peer authentication: At the end of the handshake between A and B, "A thinks that she is talking to B" and "B thinks that she is talking to A."

→ As with encryption, there are many (increasingly strong) sec. defns. here.

Handshake syntax (“One-sided authenticated key exchange”)

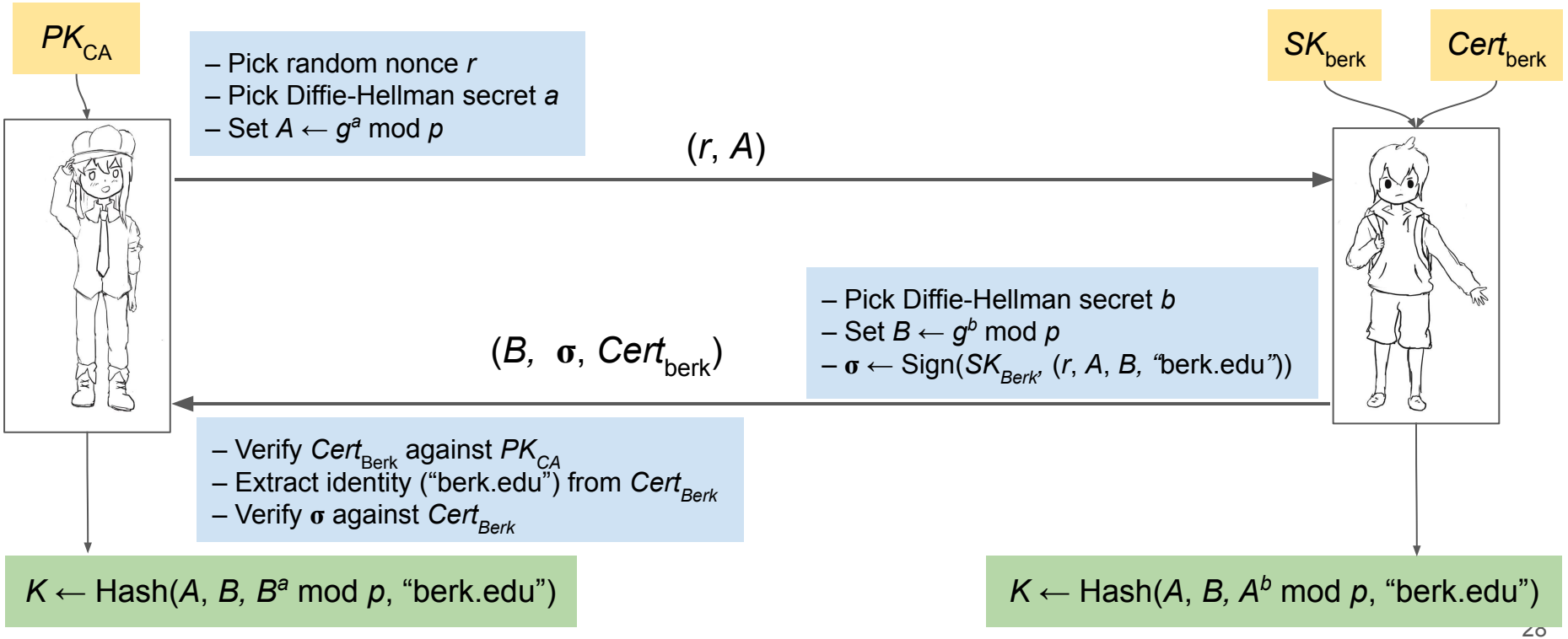


Handshake syntax (“One-sided authenticated key exchange”)



Basic handshake (weak security)

Parameters:
– Diffie-Hellman params *g* and *p*



Safe variant? Don't sign Berk's identity

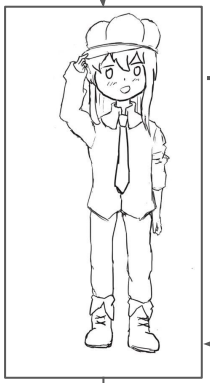
Parameters:
– Diffie-Hellman params *g* and *p*

PK_{CA}

– Pick random nonce *r*
– Pick Diffie-Hellman secret *a*
– Set $A \leftarrow g^a \bmod p$

(r, A)

SK_{berk} $Cert_{berk}$





– Pick Diffie-Hellman secret *b*
– Set $B \leftarrow g^b \bmod p$
– $\sigma \leftarrow \text{Sign}(SK_{Berk}, (r, A, B, \text{“berk.edu”}))$

$(B, \ \sigma, Cert_{berk})$

– Verify $Cert_{Berk}$ against PK_{CA}
– Extract identity (“berk.edu”) from $Cert_{Berk}$
– Verify σ against $Cert_{Berk}$

$K \leftarrow \text{Hash}(A, B, B^a \bmod p, \text{“berk.edu”})$

$K \leftarrow \text{Hash}(A, B, A^b \bmod p, \text{“berk.edu”})$

Subtle attack: “Identity misbinding”

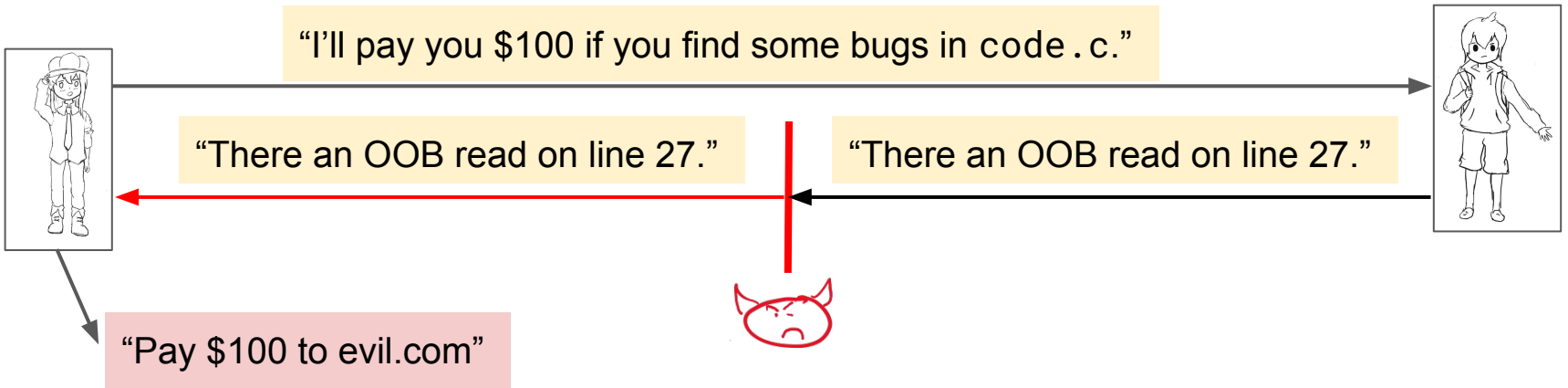
Step 1: Adversary gets a certificate $Cert_{evil}$



Step 2: Adversary interposes on handshake

Subtle attack: “Identity misbinding”

- Attack violates “peer identity” property: Each party correctly identifies the other
- BUT, Alice and Berk agree on a random session key *K* that adv doesn’t know
  - **Is there really a security problem here?**



An additional security goal: Forward security

Forward secrecy. If an attacker steals the secrets on the client or server, the attacker cannot decrypt past traffic.

How protocols achieve forward secrecy:

- Use long-term secrets only for **signing**, not Diffie-Hellman (or encryption).
- For each key exchange, generate fresh "ephemeral" DH keys: g^a and g^b
  - Alice sends Bob g^a, signed with her long term PK_{Alice}
  - Bob sends Alice g^b, signed with his long term PK_{Bob}
- Session key $K \leftarrow$ Hash(g^a, g^b, g^{ab}, <other stuff>)
- When Alice and Bob close their connection, they delete a, b, g^a, g^b, K

⇒ If Eve compromises Alice/Bob, doesn't learn any blue stuff ⇒ Can't decrypt

Even more handshake security goals…

Downgrade protection.

The parties agree on the same version of TLS that they would agree on absent an in-network attacker.

Protection against key-compromise impersonation.

If an attacker steals a client's secret key, the attacker should not be able to impersonate other servers to the client.

Protection of endpoint identities.

The public keys of the two parties should never flow over the wire in the clear.

- If client connects to website hosted on Amazon Web Services, an attacker should not be able to tell which AWS-hosted website the client is connecting to

TLS 1.3 Handshake

- If you want all the gory details, see Chapter 21.10 of the Bon[illegible]ook

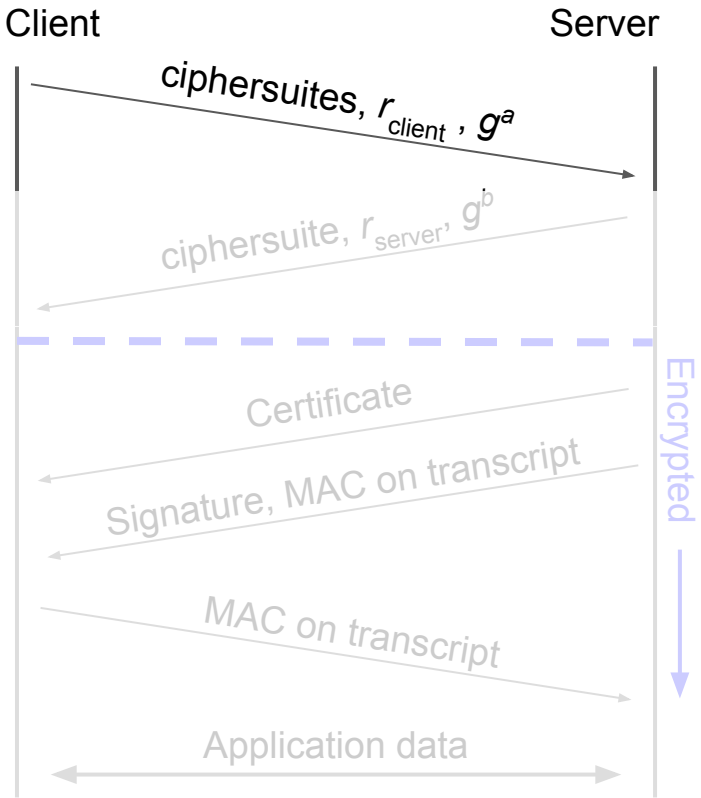
- Key differences from our basic handshake
  - Client and server negotiate which "ciphersuite" to use → flexibility
  - Server certificate travels encrypted over the wire

- Supported ciphersuites (TLS 1.3) – All use "elliptic-curve" Diffie Hellman
  - `TLS_AES_128_GCM_SHA256` = AES + GCM mode + SHA256
  - `TLS_AES_256_GCM_SHA384`
  - `TLS_CHACHA20_POLY1305_SHA256`
  - `TLS_AES_128_CCM_SHA256` = AES + Counter mode + CBC MAC + SHA256
  - `TLS_AES_128_CCM_8_SHA256`



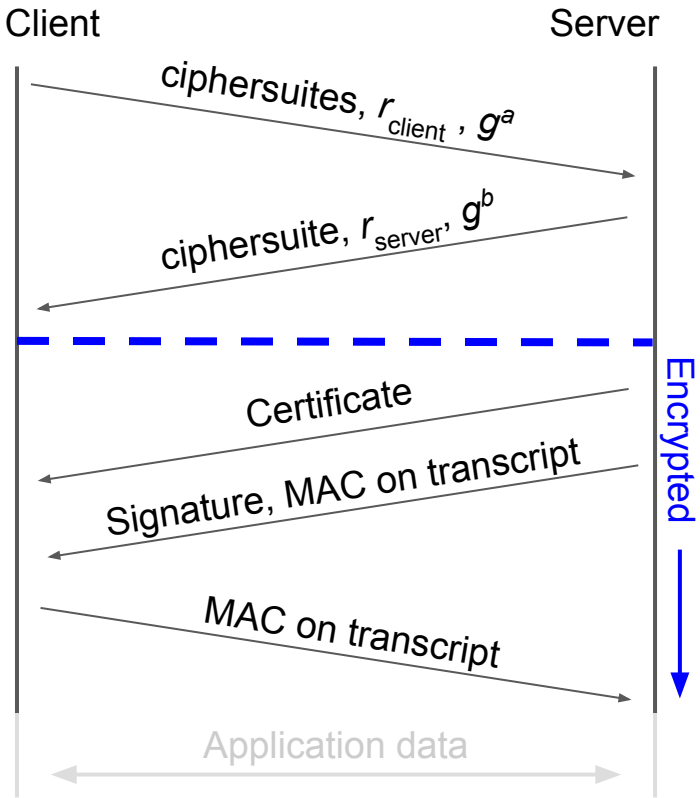
TLS Handshake Step 1: Exchange nonces and DH keys

- TCP connection already open
- The client sends **ClientHello** with
  - A nonce (“client random”)
  - Supported ciphersuites
  - Also sends: DH public key
- The server sends **ServerHello** with
  - A nonce (“server random”)
  - Chosen ciphersuite
  - Also sends: DH public key
- Both sides derive DH shared secret g^{ab}



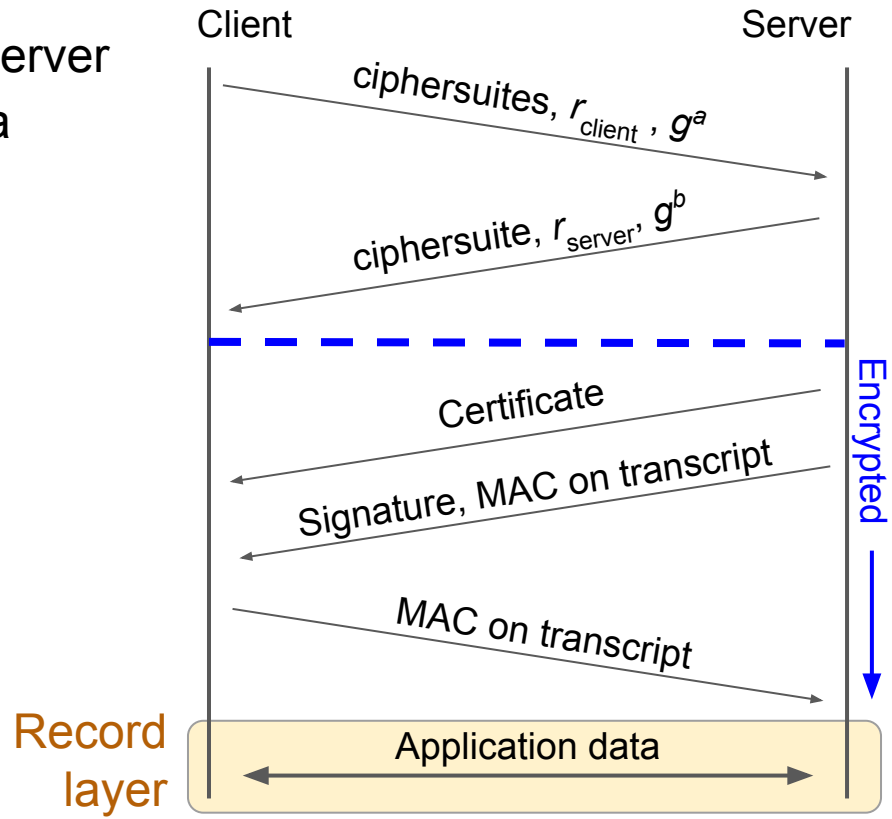
TLS Handshake Step 2: Exchange MACs

- Handshake msgs now are **encrypted**
  - Using key derived from DH shared secret g^{ab}
- Server sends certificate to client
  - [Optionally: Client sends cert to server]
- Server signs transcript with secret signing key
- Both client and server MAC transcript with a key derived from DH secret g^{ab}
  - If either MAC is invalid, output “FAIL”



TLS Handshake Step 3: Exchange application data

- After handshake finishes, client and server start sending bytes of application data
- App data = Whatever bytestream the application above wants to send
  - E.g., for HTTP: "`GET / HTTP/1.1...`"



TLS 1.2 supported many ciphersuites

NULL / export / legacy RSA, DH, DHE
TLS_NULL_WITH_NULL_NULL
TLS_RSA_WITH_NULL_MD5
TLS_RSA_WITH_NULL_SHA
TLS_RSA_EXPORT_WITH_RC4_40_MD5
TLS_RSA_WITH_RC4_128_MD5
TLS_RSA_WITH_RC4_128_SHA
TLS_RSA_EXPORT_WITH_RC2_CBC_40_MD5
TLS_RSA_WITH_IDEA_CBC_SHA
TLS_RSA_EXPORT_WITH_DES40_CBC_SHA
TLS_RSA_WITH_DES_CBC_SHA
TLS_RSA_WITH_3DES_EDE_CBC_SHA
TLS_DH_DSS_EXPORT_WITH_DES40_CBC_SHA
TLS_DH_DSS_WITH_DES_CBC_SHA
TLS_DH_DSS_WITH_3DES_EDE_CBC_SHA
TLS_DH_RSA_EXPORT_WITH_DES40_CBC_SHA
TLS_DH_RSA_WITH_DES_CBC_SHA
TLS_DH_RSA_WITH_3DES_EDE_CBC_SHA
TLS_DHE_DSS_EXPORT_WITH_DES40_CBC_SHA
TLS_DHE_DSS_WITH_DES_CBC_SHA
TLS_DHE_DSS_WITH_3DES_EDE_CBC_SHA
TLS_DHE_RSA_EXPORT_WITH_DES40_CBC_SHA
TLS_DHE_RSA_WITH_DES_CBC_SHA
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA
TLS_DH_anon_EXPORT_WITH_RC4_40_MD5
TLS_DH_anon_WITH_RC4_128_MD5
TLS_DH_anon_EXPORT_WITH_DES40_CBC_SHA
TLS_DH_anon_WITH_DES_CBC_SHA
TLS_DH_anon_WITH_3DES_EDE_CBC_SHA

Kerberos
TLS_KRB5_WITH_DES_CBC_SHA
TLS_KRB5_WITH_3DES_EDE_CBC_SHA
TLS_KRB5_WITH_RC4_128_SHA
TLS_KRB5_WITH_IDEA_CBC_SHA
TLS_KRB5_WITH_DES_CBC_MD5
TLS_KRB5_WITH_3DES_EDE_CBC_MD5
TLS_KRB5_WITH_RC4_128_MD5
TLS_KRB5_WITH_IDEA_CBC_MD5
TLS_KRB5_EXPORT_WITH_DES_CBC_40_SHA
TLS_KRB5_EXPORT_WITH_RC2_CBC_40_SHA
TLS_KRB5_EXPORT_WITH_RC4_40_SHA
TLS_KRB5_EXPORT_WITH_DES_CBC_40_MD5
TLS_KRB5_EXPORT_WITH_RC2_CBC_40_MD5
TLS_KRB5_EXPORT_WITH_RC4_40_MD5

PSK (RFC 4279 / 5487 families)
TLS_PSK_WITH_NULL_SHA
TLS_DHE_PSK_WITH_NULL_SHA
TLS_RSA_PSK_WITH_NULL_SHA
TLS_PSK_WITH_RC4_128_SHA
TLS_PSK_WITH_3DES_EDE_CBC_SHA
TLS_PSK_WITH_AES_128_CBC_SHA
TLS_PSK_WITH_AES_256_CBC_SHA
TLS_DHE_PSK_WITH_RC4_128_SHA
TLS_DHE_PSK_WITH_3DES_EDE_CBC_SHA
TLS_DHE_PSK_WITH_AES_128_CBC_SHA
TLS_DHE_PSK_WITH_AES_256_CBC_SHA
TLS_RSA_PSK_WITH_RC4_128_SHA
TLS_RSA_PSK_WITH_3DES_EDE_CBC_SHA
TLS_RSA_PSK_WITH_AES_128_CBC_SHA
TLS_RSA_PSK_WITH_AES_256_CBC_SHA
TLS_PSK_WITH_AES_128_GCM_SHA256
TLS_PSK_WITH_AES_256_GCM_SHA384
TLS_DHE_PSK_WITH_AES_128_GCM_SHA256
TLS_DHE_PSK_WITH_AES_256_GCM_SHA384
TLS_RSA_PSK_WITH_AES_128_GCM_SHA256
TLS_RSA_PSK_WITH_AES_256_GCM_SHA384
TLS_PSK_WITH_AES_128_CBC_SHA256
TLS_PSK_WITH_AES_256_CBC_SHA384
TLS_PSK_WITH_NULL_SHA256
TLS_PSK_WITH_NULL_SHA384
TLS_DHE_PSK_WITH_AES_128_CBC_SHA256
TLS_DHE_PSK_WITH_AES_256_CBC_SHA384
TLS_DHE_PSK_WITH_NULL_SHA256
TLS_DHE_PSK_WITH_NULL_SHA384
TLS_RSA_PSK_WITH_AES_128_CBC_SHA256
TLS_RSA_PSK_WITH_AES_256_CBC_SHA384
TLS_RSA_PSK_WITH_NULL_SHA256
TLS_RSA_PSK_WITH_NULL_SHA384

RSA / DH / DHE with AES and Camellia
TLS_RSA_WITH_AES_128_CBC_SHA
TLS_DH_DSS_WITH_AES_128_CBC_SHA
TLS_DH_RSA_WITH_AES_128_CBC_SHA
TLS_DHE_DSS_WITH_AES_128_CBC_SHA
TLS_DH_anon_WITH_AES_128_CBC_SHA
TLS_RSA_WITH_AES_256_CBC_SHA
TLS_DH_DSS_WITH_AES_256_CBC_SHA
TLS_DH_RSA_WITH_AES_256_CBC_SHA
TLS_DHE_DSS_WITH_AES_256_CBC_SHA
TLS_DHE_RSA_WITH_AES_256_CBC_SHA
TLS_DH_anon_WITH_AES_256_CBC_SHA
TLS_RSA_WITH_NULL_SHA256
TLS_RSA_WITH_AES_128_CBC_SHA256
TLS_RSA_WITH_AES_256_CBC_SHA256
TLS_DH_DSS_WITH_AES_128_CBC_SHA256
TLS_DH_RSA_WITH_AES_128_CBC_SHA256
TLS_DHE_DSS_WITH_AES_128_CBC_SHA256
TLS_RSA_WITH_CAMELLIA_128_CBC_SHA
TLS_DH_DSS_WITH_CAMELLIA_128_CBC_SHA
TLS_DH_RSA_WITH_CAMELLIA_128_CBC_SHA
TLS_DHE_DSS_WITH_CAMELLIA_128_CBC_SHA
TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA
TLS_DH_anon_WITH_CAMELLIA_128_CBC_SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
TLS_DH_DSS_WITH_AES_256_CBC_SHA256
TLS_DH_RSA_WITH_AES_256_CBC_SHA256
TLS_DHE_DSS_WITH_AES_256_CBC_SHA256
TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
TLS_DH_anon_WITH_AES_128_CBC_SHA256
TLS_DH_anon_WITH_AES_256_CBC_SHA256
TLS_RSA_WITH_CAMELLIA_256_CBC_SHA
TLS_DH_DSS_WITH_CAMELLIA_256_CBC_SHA
TLS_DH_RSA_WITH_CAMELLIA_256_CBC_SHA
TLS_DHE_DSS_WITH_CAMELLIA_256_CBC_SHA
TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA
TLS_DH_anon_WITH_CAMELLIA_256_CBC_SHA

SEED
TLS_RSA_WITH_SEED_CBC_SHA
TLS_DH_DSS_WITH_SEED_CBC_SHA
TLS_DH_RSA_WITH_SEED_CBC_SHA
TLS_DHE_DSS_WITH_SEED_CBC_SHA
TLS_DHE_RSA_WITH_SEED_CBC_SHA
TLS_DH_anon_WITH_SEED_CBC_SHA

AES-GCM (non-ECC)
TLS_RSA_WITH_AES_128_GCM_SHA256
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
TLS_DH_RSA_WITH_AES_128_GCM_SHA256
TLS_DH_RSA_WITH_AES_256_GCM_SHA384
TLS_DHE_DSS_WITH_AES_128_GCM_SHA256
TLS_DHE_DSS_WITH_AES_256_GCM_SHA384
TLS_DH_DSS_WITH_AES_128_GCM_SHA256
TLS_DH_DSS_WITH_AES_256_GCM_SHA384
TLS_DH_anon_WITH_AES_128_GCM_SHA256
TLS_DH_anon_WITH_AES_256_GCM_SHA384

SRP
TLS_SRP_SHA_WITH_3DES_EDE_CBC_SHA
TLS_SRP_SHA_RSA_WITH_3DES_EDE_CBC_SHA
TLS_SRP_SHA_DSS_WITH_3DES_EDE_CBC_SHA
TLS_SRP_SHA_WITH_AES_128_CBC_SHA
TLS_SRP_SHA_RSA_WITH_AES_128_CBC_SHA
TLS_SRP_SHA_DSS_WITH_AES_128_CBC_SHA
TLS_SRP_SHA_WITH_AES_256_CBC_SHA
TLS_SRP_SHA_RSA_WITH_AES_256_CBC_SHA
TLS_SRP_SHA_DSS_WITH_AES_256_CBC_SHA

ECC / ECDH / ECDHE
TLS_ECDH_ECDSA_WITH_NULL_SHA
TLS_ECDH_ECDSA_WITH_RC4_128_SHA
TLS_ECDH_ECDSA_WITH_3DES_EDE_CBC_SHA
TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA
TLS_ECDH_ECDSA_WITH_AES_256_CBC_SHA
TLS_ECDHE_ECDSA_WITH_NULL_SHA
TLS_ECDHE_ECDSA_WITH_RC4_128_SHA
TLS_ECDHE_ECDSA_WITH_3DES_EDE_CBC_SHA
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA
TLS_ECDH_RSA_WITH_NULL_SHA
TLS_ECDH_RSA_WITH_RC4_128_SHA
TLS_ECDH_RSA_WITH_3DES_EDE_CBC_SHA
TLS_ECDH_RSA_WITH_AES_128_CBC_SHA
TLS_ECDH_RSA_WITH_AES_256_CBC_SHA
TLS_ECDHE_RSA_WITH_NULL_SHA
TLS_ECDHE_RSA_WITH_RC4_128_SHA
TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
TLS_ECDH_anon_WITH_NULL_SHA

TLS 1.2 supported many ciphersuites

LS_PSK_WITH_CAMELLIA_128_CBC_SHA256
TLS_PSK_WITH_CAMELLIA_256_CBC_SHA384
TLS_DHE_PSK_WITH_CAMELLIA_128_CBC_SHA256
TLS_DHE_PSK_WITH_CAMELLIA_256_CBC_SHA384
TLS_RSA_PSK_WITH_CAMELLIA_128_CBC_SHA256
TLS_RSA_PSK_WITH_CAMELLIA_256_CBC_SHA384
TLS_ECDHE_PSK_WITH_CAMELLIA_128_CBC_SHA256
TLS_ECDHE_PSK_WITH_CAMELLIA_256_CBC_SHA384

ARIA
TLS_RSA_WITH_ARIA_128_CBC_SHA256
TLS_RSA_WITH_ARIA_256_CBC_SHA384
TLS_DH_DSS_WITH_ARIA_128_CBC_SHA256
TLS_DH_DSS_WITH_ARIA_256_CBC_SHA384
TLS_DH_RSA_WITH_ARIA_128_CBC_SHA256
TLS_DH_RSA_WITH_ARIA_256_CBC_SHA384
TLS_DHE_DSS_WITH_ARIA_128_CBC_SHA256
TLS_DHE_DSS_WITH_ARIA_256_CBC_SHA384
TLS_DHE_RSA_WITH_ARIA_128_CBC_SHA256
TLS_DHE_RSA_WITH_ARIA_256_CBC_SHA384
TLS_DH_anon_WITH_ARIA_128_CBC_SHA256
TLS_DH_anon_WITH_ARIA_256_CBC_SHA384
TLS_ECDHE_ECDSA_WITH_ARIA_128_CBC_SHA256
TLS_ECDHE_ECDSA_WITH_ARIA_256_CBC_SHA384
TLS_ECDH_ECDSA_WITH_ARIA_128_CBC_SHA256
TLS_ECDH_ECDSA_WITH_ARIA_256_CBC_SHA384
TLS_ECDHE_RSA_WITH_ARIA_128_CBC_SHA256
TLS_ECDHE_RSA_WITH_ARIA_256_CBC_SHA384
TLS_ECDH_RSA_WITH_ARIA_128_CBC_SHA256
TLS_ECDH_RSA_WITH_ARIA_256_CBC_SHA384
TLS_RSA_WITH_ARIA_128_GCM_SHA256
TLS_RSA_WITH_ARIA_256_GCM_SHA384
TLS_DHE_RSA_WITH_ARIA_128_GCM_SHA256
TLS_DHE_RSA_WITH_ARIA_256_GCM_SHA384
TLS_DH_RSA_WITH_ARIA_128_GCM_SHA256
TLS_DH_RSA_WITH_ARIA_256_GCM_SHA384
TLS_DHE_DSS_WITH_ARIA_128_GCM_SHA256
TLS_DHE_DSS_WITH_ARIA_256_GCM_SHA384
TLS_DH_DSS_WITH_ARIA_128_GCM_SHA256
TLS_DH_DSS_WITH_ARIA_256_GCM_SHA384
TLS_DH_anon_WITH_ARIA_128_GCM_SHA256
TLS_DH_anon_WITH_ARIA_256_GCM_SHA384
TLS_ECDHE_ECDSA_WITH_ARIA_128_GCM_SHA256

TLS_RSA_PSK_WITH_ARIA_256_CBC_SHA384
TLS_PSK_WITH_ARIA_128_GCM_SHA256
TLS_PSK_WITH_ARIA_256_GCM_SHA384
TLS_DHE_PSK_WITH_ARIA_128_GCM_SHA256
TLS_DHE_PSK_WITH_ARIA_256_GCM_SHA384
TLS_RSA_PSK_WITH_ARIA_128_GCM_SHA256
TLS_RSA_PSK_WITH_ARIA_256_GCM_SHA384
TLS_ECDHE_PSK_WITH_ARIA_128_CBC_SHA256
TLS_ECDHE_PSK_WITH_ARIA_256_CBC_SHA384

ChaCha20-Poly1305
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256
TLS_PSK_WITH_CHACHA20_POLY1305_SHA256
TLS_ECDHE_PSK_WITH_CHACHA20_POLY1305_SHA256
TLS_DHE_PSK_WITH_CHACHA20_POLY1305_SHA256
TLS_RSA_PSK_WITH_CHACHA20_POLY1305_SHA256

CCM / CCM_8
TLS_RSA_WITH_AES_128_CCM
TLS_RSA_WITH_AES_256_CCM
TLS_DHE_RSA_WITH_AES_128_CCM
TLS_DHE_RSA_WITH_AES_256_CCM
TLS_RSA_WITH_AES_128_CCM_8
TLS_RSA_WITH_AES_256_CCM_8
TLS_DHE_RSA_WITH_AES_128_CCM
TLS_DHE_RSA_WITH_AES_256_CCM
TLS_DHE_RSA_WITH_AES_128_CCM_8
TLS_DHE_RSA_WITH_AES_256_CCM_8
TLS_PSK_WITH_AES_128_CCM
TLS_PSK_WITH_AES_256_CCM
TLS_DHE_PSK_WITH_AES_128_CCM
TLS_DHE_PSK_WITH_AES_256_CCM
TLS_PSK_WITH_AES_128_CCM_8
TLS_PSK_WITH_AES_256_CCM_8
TLS_PSK_DHE_WITH_AES_128_CCM_8
TLS_PSK_DHE_WITH_AES_256_CCM_8
TLS_ECDHE_ECDSA_WITH_AES_128_CCM
TLS_ECDHE_ECDSA_WITH_AES_256_CCM
TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8
TLS_ECDHE_ECDSA_WITH_AES_256_CCM_8

TLS_ECDH_anon_WITH_3DES_EDE_CBC_SHA
TLS_ECDH_anon_WITH_AES_128_CBC_SHA
TLS_ECDH_anon_WITH_AES_256_CBC_SHA
TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA256
TLS_ECDH_ECDSA_WITH_AES_256_CBC_SHA384
TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
TLS_ECDH_RSA_WITH_AES_128_CBC_SHA256
TLS_ECDH_RSA_WITH_AES_256_CBC_SHA384
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
TLS_ECDH_ECDSA_WITH_AES_128_GCM_SHA256
TLS_ECDH_ECDSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDH_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDH_RSA_WITH_AES_256_GCM_SHA384

ECC + Camellia
TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256
TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384
TLS_ECDH_ECDSA_WITH_CAMELLIA_128_CBC_SHA256
TLS_ECDH_ECDSA_WITH_CAMELLIA_256_CBC_SHA384
TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256
TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384
TLS_ECDH_RSA_WITH_CAMELLIA_128_CBC_SHA256
TLS_ECDH_RSA_WITH_CAMELLIA_256_C
TLS_ECDHE_ECDSA_WITH_CAMELLIA_
TLS_ECDHE_ECDSA_WITH_CAMELLIA
TLS_ECDH_ECDSA_WITH_CAMELLIA_
TLS_ECDH_ECDSA_WITH_CAMELLIA_
TLS_ECDHE_RSA_WITH_CAMELLIA_12
TLS_ECDHE_RSA_WITH_CAMELLIA_25
TLS_ECDH_RSA_WITH_CAMELLIA_128
TLS_ECDH_RSA_WITH_CAMELLIA_256
TLS_PSK_WITH_CAMELLIA_128_GCM_
TLS_PSK_WITH_CAMELLIA_256_GCM_
TLS_DHE_PSK_WITH_CAMELLIA_128_
TLS_DHE_PSK_WITH_CAMELLIA_256_
TLS_RSA_PSK_WITH_CAMELLIA_128_GC
TLS_RSA_PSK_WITH_CAMELLIA_256_GCM_SHA384

TLS_ECDHE_ECDSA_WITH_ARIA_256_GCM_SHA3
TLS_ECDH_ECDSA_WITH_ARIA_128_GCM_SHA25
TLS_ECDH_ECDSA_WITH_ARIA_256_GCM_SHA38
TLS_ECDHE_RSA_WITH_ARIA_128_GCM_SHA256
TLS_ECDHE_RSA_WITH_ARIA_256_GCM_SHA384
TLS_ECDH_RSA_WITH_ARIA_128_GCM_SHA256
TLS_ECDH_RSA_WITH_ARIA_256_GCM_SHA384
TLS_PSK_WITH_ARIA_128_CBC_SHA256
TLS_PSK_WITH_ARIA_256_CBC_SHA384
TLS_DHE_PSK_WITH_ARIA_128_CBC_SHA256
TLS_DHE_PSK_WITH_ARIA_256_CBC_SHA384
TLS_RSA_PSK_WITH_ARIA_128_CBC_SHA256

ECDHE_PSK with AES-GCM / AES-CCM
TLS_ECDHE_PSK_WITH_AES_128_GCM_SHA256
TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384
TLS_ECDHE_PSK_WITH_AES_128_CCM_SHA256
TLS_ECDHE_PSK_WITH_AES_256_CCM_SHA384

Lesson: Many options
⇒ More code
⇒ more edge cases
⇒ Bad for security

TLS Record Layer



https://openclipart.org/detail/319127/lady-with-record-player

TLS has two sub-protocols





Handshake = establish shared secret key

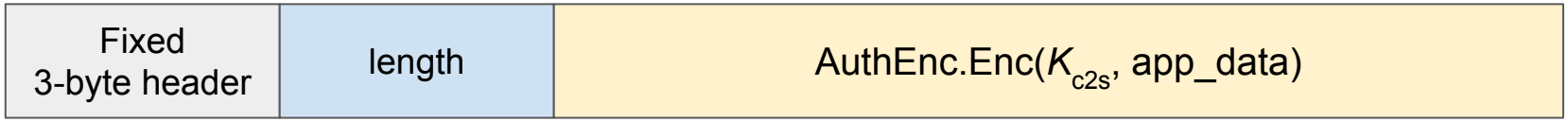
Key *K*

Key *K*

Record Layer = bulk data transfer

TLS 1.3 record layer

- Blissfully simple – all of the complexity is in the handshake
- Each party sends data in TLS "records" with <= 16 KB of data each
- Both parties derive K_{c2s} and K_{s2c} from the handshake "master secret"
- Each side encrypts data using an authenticated-encryption scheme
  - Plus one extra tweak to save some bandwidth



TLS: Costs

- Handshake
  - Diffie-Hellman key exchange
  - Plus a signature / sig. Verification
  - Plus a MAC and some hashes to derive keys
- Record layer
  - Modern hardware has dedicated support for symmetric-key cryptography – not a bottleneck
  - For IoT devices, encryption can still be costly
- Big annoyance: Latency = Extra round trip before first byte of data
  - Must perform the entire TLS handshake before sending the first message
  - TLS 1.3 supports dangerous “0RTT” mode – data on “first flight” → Vulnerable to replay

TLS: Summary

- TLS = TCP + authenticated encryption
  - The protocol you want for sending streams of data over the Internet
  - Uses everything we have learned so far! Encryption, MAC, Signatures, DH key exchange

- Security properties = Subtle and hard to get right
  - TLS 1.3 underwent four years of iteration – ***29*** draft protocols
  - Lots of bugs found even late in the game
  - Formal verification and crypto security proofs played a central role

- Next time
  - Attacks on prior versions of TLS
  - What TLS doesn't protect you against